

INFORME TÉCNICO INTEGRAL: VISIÓN OS (VISION COGNITIVE AGENT)

Versión: 1.0
 Fecha: 28 de Julio de 2026
 Proyecto: **vision-cognitive-agent** (Visión OS)
 Ubicación: **material_complementario/informe_general_vision_os.md**
 Estado: ■ Corregido tras auditoría de consistencia

1. RESUMEN EJECUTIVO

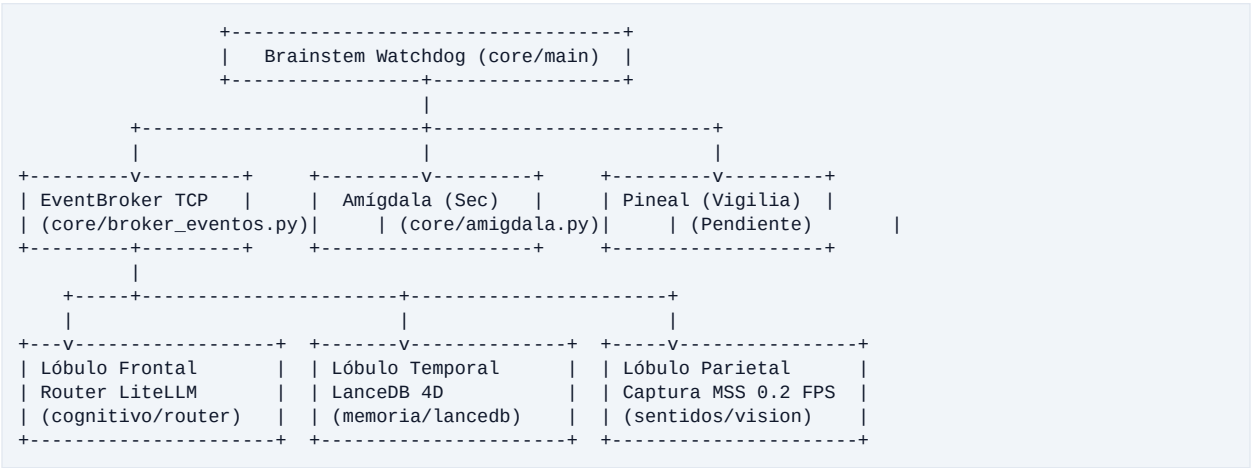
Visión OS es un Sistema Operativo Cognitivo Neuro-Mimético diseñado para la ejecución autónoma de agentes de inteligencia artificial. A diferencia de las arquitecturas secuenciales monolíticas tradicionales, Visión OS opera como un **ecosistema distribuido de daemons asíncronos en tiempo real**, interconectados mediante un bus central de eventos sobre TCP Sockets y supervisados por un daemon Watchdog autorregulable (*BrainstemWatchdog*).

Hitos Clave del Sistema:

- **Fases 0 a 6 (100% Completadas):** Desde la infraestructura base y bus TCP hasta la visión parietal, oídos activos (*faster-whisper*), ejecución sandbox en Docker, y módulo creativo ComfyUI.
- **Backend v2 & Memoria 4D:** Migración a esquema vectorial 4D en LanceDB con búsqueda híbrida (Full-Text Search + Dense Vectors) e integración de *Progressive RAG*.
- **Visión Studio (Frontend HUD):** Dashboard interactivo en React + TypeScript + Vite, con panel de control para binarios de **llama.cpp**, telemetría de hardware en tiempo real (NVIDIA RTX 5060 Ti, CPU, RAM, VRAM), soporte para MTP Speculative Decoding y sistema de notificaciones y tooltips interactivos.
- **Calidad y CI/CD:** Integración de Pytest (matriz Python 3.10-3.12), linter Ruff y GitHub Actions, garantizando máxima seguridad en el manejo de credenciales mediante **.env**.

2. ARQUITECTURA DEL SISTEMA (ANATOMÍA NEURO-MIMÉTICA)

Visión OS mapea regiones del cerebro humano a componentes de software modulares:



2.1 Sistemas Vitales (core/)

1. **Tallo Cerebral (core/main.py):** Watchdog principal que gestiona el ciclo de vida de los daemons. Monitorea salud, encodings y reinicia procesos caídos.
2. **Corazón (core/broker_eventos.py):** Bus TCP asíncrono (**asyncio**) a 127.0.0.1:5000. Soporta suscripciones con comodines (*), broadcast de mensajes ("Glóbulos Rojos") y purga de colas en emergencia.
3. **Amígdala (core/amigdala.py):** Firewall perimetral que intercepta prompts para evitar inyección de código y comandos peligrosos (**rm -rf, sudo**), activando el botón de pánico del sistema.
4. **Glándula Pineal (■■ Pendiente - no implementado):** Planeado para controlar la energía del agente y la transición entre ciclos de Vigilia (Día) y Consolidación (Noche). Actualmente no implementado como archivo independiente.

2.2 Procesamiento y Consciencia (cognitivo/)

1. **Lóbulo Frontal (cognitivo/llm_router.py):** Enrutador inteligente ("Crazy Router") que clasifica tareas por nivel de esfuerzo (**esfuerzo_bajo, esfuerzo_medio, esfuerzo_alto**), delegando en LiteLLM con fallbacks automáticos (OpenRouter / APIs Cloud -> Ollama / Llama.cpp local).
2. **Hemisferio Izquierdo (cognitivo/ejecutor_izquierdo.py):** Ejecución analítica y procedural con agentes MCP y contenedor **Docker Sandbox**.
3. **Hemisferio Derecho (cognitivo/contexto_derecho.py):** Monitoreo del contexto visuo-espacial del usuario y estado afectivo dinámico (**emociones.json**).
4. **Gestor Llama.cpp (cognitivo/gestor_llamacpp.py):** Módulo de control del servidor **llama-server.exe**, telemetría de hardware (**nvidia-smi, psutil**) y ajuste de MTP Speculative Decoding (**--spec-draft-n-max**).

2.3 Memoria y Archivo (memoria/)

1. **Lóbulo Temporal (memoria/lancedb_manager.py):** Base vectorial 4D en LanceDB con embeddings locales de 1024 dimensiones (**BAAI/bge-m3**).
2. **Hipocampo (memoria/hipocampo.py):** Consolidación asíncrona post-proceso. Sintetiza conversaciones/logs en 3 viñetas y 5 etiquetas antes de indexar.
3. **Cargador de Datasets (memoria/cargador_datasets.py):** Carga y preprocesamiento de datasets externos para aumentar la base de conocimiento.
4. **Cerebelo / Renderizador HDD (■■ Pendiente):** Motor planeado de gestión física Hot/Cold Tiering entre SSD y HDD. No implementado actualmente.
5. **Explorador Tavily (■■ Pendiente):** Agente de investigación web planeado para resolver vacíos de información. La funcionalidad de búsqueda web parcial está en **cognitivo/web_search.py**.

2.4 Percepción y Sentidos (sentidos/)

1. **Lóbulo Parietal (sentidos/vision_parietal.py):** Captura continua de pantalla a 0.2 FPS mediante **mss** y filtrado por diferencia de thumbnails 32x32 (Pillow).
2. **Lóbulo Occipital / Imaginación (sentidos/imaginacion_occipital.py):** Generación multimedia vía ComfyUI local.
3. **Oído Parietal (sentidos/oído_parietal.py):** Transcripción de audio en tiempo real con **faster-whisper**.
4. **Sistema Periférico (sentidos/sistema_periferico.py):** API FastAPI para recepción de webhooks e Ingestión Multimodal (PDF, CSV, Audio, Imagen, Texto) vía **POST /api/memoria/aprender**.

2.5 Módulos Existentes No Documentados en el Mapa Original

El proyecto ha evolucionado con módulos adicionales no cubiertos en la arquitectura neuro-mimética original:

- 1. **Habla Parietal (sentidos/habla_parietal.py)**: Síntesis y procesamiento de voz (TTS), complementa al Oído Parietal para cerrar el ciclo auditivo completo.
- 2. **Calibrador de Audio (sentidos/calibrador_audio.py)**: Calibración automática de dispositivos de audio y niveles de sensibilidad del micrófono.
- 3. **Gestor de Modelos Locales (cognitivo/gestor_modelos_locales.py)**: Gestión de descarga, carga y selección de modelos locales complementarios al router LiteLLM.
- 4. **Orquestador Graph (cognitivo/orquestador_graph.py)**: Orquestador basado en LangGraph que gestiona flujos de trabajo cognitivos complejos.
- 5. **Web Search (cognitivo/web_search.py)**: Módulo de búsqueda web independiente que complementa (sin reemplazar) al planeado Explorador Tavily.
- 6. **Protocolo de Intriga (cognitivo/protocolo_intriga.py)**: Sistema de activación contextual y descubrimiento progresivo de capacidades del agente.
- 7. **Distrito de Agentes (cognitivo/distrito_agentes/)**: Ecosistema de agentes especializados (docker, git, npm, pip, cargo, go, kubectl, gcloud, python) con herramientas de ejecución autónoma.
- 8. **Esquemas Compartidos (core/schemas.py)**: Definiciones de tipos y schemas Pydantic compartidos entre todos los módulos del core.

■ 3. LÓGICA MATEMÁTICA Y MOTOR DE MEMORIA 4D

3.1 Estructura Espacial Fractal (Base-3)

El espacio de memoria vectorial no es plano, sino un hipercubo fractal anidado de $3 \times 3 \times 3$ (Base-3):

$$N(n) = 27^n \implies N(4) = 27^4 = 531,441 \text{ bloques lógicos}$$

Las coordenadas globales (X_g, Y_g, Z_g, W_g) a nivel de profundidad n se obtienen mediante:

$$X_g = \sum_{i=1}^n x_i \cdot 3^{n-i}, \quad Y_g = \sum_{i=1}^n y_i \cdot 3^{n-i}, \quad Z_g = \sum_{i=1}^n z_i \cdot 3^{n-i}$$

3.2 Significado de los Ejes Espaciales:

- **Eje X / Y**: Cercanía Semántica (Similitud del Coseno entre vectores densos).
- **Eje Z**: Temperatura (Frecuencia de consulta y vigencia temporal).
- **Eje W**: Estado Afectivo / Contextual del agente.

3.3 Jerarquía de Almacenamiento y Evicción Automática

- **Caliente (SSD - \$100\textcircled{C}\$ a \$50\textcircled{C}\$)**: Memoria de trabajo del día actual.
- **Tibio (SSD/HDD - \$49\textcircled{C}\$ a \$20\textcircled{C}\$)**: Punteros LanceDB activos con binarios pesados en HDD.
- **Frío (HDD - \$<20\textcircled{C}\$)**: Historial antiguo comprimido y resúmenes ejecutivos.
- **Regla de Evicción (High-Water Mark)**: Si el uso del SSD alcanza el **85%**, el Cerebelo migra automáticamente los recuerdos más fríos al HDD hasta restablecer la ocupación al **60%**.

■ 4. FRONTEND Y UX/UI: VISIÓN STUDIO

Ubicado en `vision_studio/`, **Visión Studio** es la interfaz gráfica cyberpunk de control de Visión OS:

- **Tecnologías:** React 18, TypeScript, Vite, Tailwind CSS, Lucide React Icons.
- **LlamaCppPanel (`LlamaCppPanel.tsx`):**
 - Selector interactivo de ruta de binarios de `llama.cpp`.
 - Dashboard de telemetría en tiempo real: Uso CPU, RAM, espacio en discos, VRAM GPU (RTX 5060 Ti) y Temperatura.
 - Generador dinámico de scripts `.bat` para arranque acelerado.
 - Slider para ajuste de MTP Speculative Decoding (`--spec-draft-n-max`).
- **Sistema UX Amigable:**
 - **Notificaciones Toast (`useNotificationStore.ts`, `ToastContainer.tsx`):** Mensajes emergentes con código de colores según tipo de evento (Éxito ■, Error ■, Advertencia ■, Info ■).
 - **Tooltips Explicativos (`Tooltip.tsx`):** Definiciones emergentes para parámetros complejos (`-ngl`, `-c`, `Flash Attention`, `KV Cache`).
 - **Sidebar Chat (`SidebarChat.tsx`):** Indicador continuo de tokens por segundo (t/s), contador de context window y bloque colapsable para inspección de razonamiento interno (`<think>`).

■ ■ 5. CALIDAD DE CÓDIGO, SEGURIDAD Y CI/CD

1. **Gestión Estricta de Credenciales:** Limpieza total de API keys hardcodeadas. Uso obligatorio de `.env` interpolado en archivos de configuración (`llm_router.yaml`, `model_registry.json`).
2. **Git Workflow:** Desarrollo sobre ramas `feat/*`, `fix/*`, `release/*` partiendo de `develop`. Commits siguiendo el estándar Conventional Commits (`feat:`, `fix:`, `chore:`).
3. **Pipeline de Integración Continua (`.github/workflows/ci.yaml`):**
 - **Security Scan:** Auditoría de secretos y dependencias.
 - **Ruff Lint & Format:** Verificación de código limpio Python 3.10+.
 - **Pytest Matrix:** Ejecución automatizada de tests en Python 3.10, 3.11 y 3.12.

■ 6. MATRIZ DE ESTADO Y PRÓXIMOS PASOS

Componente	Estado	Notas / Próxima Iteración
Core Watchdog & EventBroker TCP	■ 100%	Operativo y estable en puerto 5000
Amígdala	■ 100%	Filtro de seguridad e higienización activo
Glándula Pineal	■ 0%	No implementado. Planeado para ciclo Vigilia/Sueño
LiteLLM Enrutador & Gestor Llama.cpp	■ 100%	Integrado con MTP y telemetría de hardware
Gestor Modelos Locales	■ 100%	Descarga y selección de modelos locales
Orquestador LangGraph	■ 100%	Flujos cognitivos complejos
Protocolo de Intriga	■ 100%	Descubrimiento progresivo de capacidades

Distrito de Agentes	■ 100%	9 agentes especializados (docker, git, npm, pip, cargo, go, kubectl, gcloud, python)
Memoria LanceDB 4D & Hipocampo	■ 100%	Progressive RAG + Cargador de datasets
Cerebelo / Renderizador HDD	■ 0%	No implementado. Planeado para Hot/Cold Tiering SSD/HDD
Explorador Tavily	■ 0%	No implementado. Búsqueda web parcial via cognitivo/web_search.py
Sentidos (Visión, Audio, Webhooks)	■ 100%	Ingestión Multimodal (PDF, CSV, Audio)
Habla Parietal (TTS)	■ 100%	Síntesis de voz, complemento del Oído Parietal
Calibrador de Audio	■ 100%	Calibración automática de dispositivos de audio
Visión Studio HUD UI	■ 90%	Faltan terminal Xterm.js y Monaco Editor

Informe generado automáticamente por el Arquitecto de Visión OS para preservación en [material_complementario](#).